ARKA-Lite

Proyecto final · Upskilling Backend Java

Una UI y dos microservicios Spring Boot: lo que aprendimos y cómo funciona.

RECORRIDO RESUMEN · CONTENIDO Y GUION

PARTES

1. **Fundamentos** Spring Boot, REST, reglas de negocio, pruebas y Git.
2. **Arquitectura** Microservicios, puertos y adaptadores, y un contrato por eventos.
3. **Base de datos** Una base por servicio, un esquema que se cuida solo y datos consistentes.
4. **Contenedores y Kubernetes** El mismo sistema en cualquier computadora y un clúster que se repara solo.
5. **CI/CD** Integración y entrega continua: cada cambio se prueba y se empaqueta solo.
6. **Observabilidad** Logs centralizados, trazas entre servicios y salud de cada componente.
7. **Documentación** Contratos que salen del código y documentación que cambia con él.
8. **La UI como laboratorio** No es parte del entregable: es el banco de pruebas para ver cómo se comporta el sistema, sobre todo cuando algo falla.

Generado desde docs/presentation-data/contenido. Cada lámina tiene: lo que se ve, el diagrama detallado cuando lo hay, y el guion: qué decir, qué hacer, si preguntan y para pasar.

Apertura

1. Del endpoint al clúster

EN PANTALLA

- Lo que aprendimos construyendo dos microservicios Java, de punta a punta.

QUÉ DECIR

“Buenos días. Hoy les presentamos ARKA-Lite, el proyecto final del curso Upskilling Backend Java.”

“Empezamos escribiendo un endpoint y terminamos con un sistema de dos microservicios corriendo en Kubernetes, con su pipeline, su base de datos y su observabilidad. No les vamos a contar el temario: les vamos a contar qué construimos, qué medimos y qué aprendimos.”

QUÉ HACER (NO SE DICE)

- Antes de empezar, abrir la UI en http://localhost:8088 y comprobar que los dos servicios aparecen en verde.

PARA PASAR

“Primero, el proyecto en números.”

2. ARKA-Lite en números

EN PANTALLA · NÚMEROS GRANDES

- 2 microservicios: Spring Boot 4 y Java 26, cada uno con su PostgreSQL.
- 11 endpoints REST: Documentados con OpenAPI y Swagger.
- 67 pruebas automáticas: 28 en solicitudes, 10 en notificaciones, 29 en la UI.
- 8 contenedores: Servicios, bases, UI y logs con un solo comando.
- <= 5 s para entregar un aviso: El relevador revisa los pendientes cada 5 segundos.
- 0 avisos perdidos o duplicados: Aunque el otro servicio esté caído o el evento llegue dos veces.

IDEAS CLAVE

- Dos microservicios con su propia base
- Un solo comando levanta todo
- Cada cambio pasa por pruebas automáticas

QUÉ DECIR

“ARKA-Lite hace algo sencillo a propósito: registra solicitudes, como una incidencia o un pedido de cambio, y avisa cuando una solicitud se envía. El problema es pequeño para que el foco esté en cómo se construye, se despliega y se opera.”

“Son dos microservicios con Spring Boot, once endpoints documentados, sesenta y siete pruebas automáticas y ocho contenedores que se levantan con un solo comando.”

“Y los dos números que más nos importan: un aviso se entrega en cinco segundos o menos, y no se pierde ni se duplica ninguno, aunque el otro servicio esté caído. Más adelante les mostramos cómo lo logramos.”

PARA PASAR

“Antes de entrar, ocho palabras que vamos a usar mucho.”

---

3. Ocho conceptos, en una frase

EN PANTALLA · TARJETAS

- Microservicio: Una aplicación pequeña, con una sola tarea y su propia base de datos.
- API REST: La puerta para pedirle algo a una aplicación por HTTP, como abrir una URL.
- Arquitectura hexagonal: El negocio al centro; la base y la web se conectan por “enchufes”.
- ORM (JPA): Traduce entre clases Java y tablas: escribe el SQL rutinario por nosotros.
- Docker: Empaqueta la aplicación con todo lo que necesita: corre igual en cualquier lado.
- Kubernetes: Administra esos paquetes: los arranca, los vigila y los reinicia si fallan.
- CI/CD: Un robot que compila, prueba y empaqueta cada cambio, sin pasos manuales.
- Observabilidad: Logs y salud de todo el sistema en un solo lugar.

IDEAS CLAVE

- Microservicio y API REST
- Arquitectura hexagonal y ORM
- Docker y Kubernetes
- CI/CD y observabilidad

QUÉ DECIR

“Sabemos que muchos de ustedes trabajan sobre todo con la base de datos y procedimientos almacenados, así que antes de entrar les dejamos ocho conceptos en una frase.”

“Un microservicio es una aplicación pequeña con una sola tarea y su propia base. Una API REST es la puerta para pedirle cosas por HTTP. La arquitectura hexagonal pone el negocio al centro y conecta la base y la web como si fueran enchufes. Y un ORM traduce entre clases Java y tablas, para no escribir a mano el SQL de todos los días.”

“Docker empaqueta la aplicación con todo lo que necesita; Kubernetes administra esos paquetes. CI/CD es un robot que prueba cada cambio. Y observabilidad es poder ver qué pasa adentro. Cada uno lo vamos a ver con un ejemplo.”

SI PREGUNTAN

¿Esto reemplaza a los procedimientos almacenados?

No necesariamente. Mueve las reglas del negocio al código; el SQL pesado o los reportes pueden seguir en la base.

PARA PASAR

“Este es el recorrido.”

1 · Fundamentos

Spring Boot, REST, reglas de negocio, pruebas y Git.

4. Cómo funciona Spring Boot

EN PANTALLA · FLUJO PASO A PASO

- 1. main(): SpringApplication.run arranca la aplicación.
- 2. Autoconfiguración: Ve web, JPA y PostgreSQL y los configura solos.
- 3. Beans e inyección: Crea cada pieza y le pasa lo que necesita.
- 4. Servidor embebido: Tomcat queda escuchando en el puerto 8080.
- 5. Controlador: Recibe el JSON, llama al negocio y responde.

IDEAS CLAVE

- Un main arranca todo
- Se configura solo según las librerías
- Crea las piezas y las conecta por el constructor
- Trae su propio servidor web

Comparación: Como un contratista de obra: le decimos qué piezas hay y él las arma, las conecta y abre la puerta.

QUÉ DECIR

“Antes de Spring Boot, levantar una aplicación Java web significaba instalar un servidor, configurar la conexión a la base y conectar cada clase a mano. Spring Boot hace ese trabajo.”

“Todo empieza en un main. Spring Boot mira qué librerías tenemos, por ejemplo web, JPA y el driver de PostgreSQL, y las configura solo. Luego crea cada pieza de la aplicación, lo que en Spring se llama un bean, y le pasa por el constructor lo que necesita: eso es la inyección de dependencias.”

“Al final arranca su propio servidor web, Tomcat, en el puerto 8080. Cuando llega una petición, el controlador la recibe, llama al negocio y responde. Nosotros solo escribimos el negocio y decimos qué piezas hay.”

QUÉ HACER (NO SE DICE)

- Mostrar: Ejecutar ./mvnw clean verify dentro de servicio-solicitudes. Debe verse: BUILD SUCCESS y el resumen de pruebas.

SI PREGUNTAN

¿Qué es un bean?

Un objeto que Spring crea y administra, y que puede pasarle a otros por el constructor.

PARA PASAR

“Con la aplicación arriba, veamos cómo responde la API.”

---

5. Código mantenible: buenas prácticas

EN PANTALLA · TABLA DE TRES COLUMNAS

- Práctica: Una responsabilidad por clase · Cómo la aplicamos: El controlador traduce, el servicio decide, el adaptador guarda · Por qué ayuda a mantener: Un cambio toca un solo lugar
- Práctica: Inyección por constructor · Cómo la aplicamos: Cada pieza recibe sus dependencias; nadie hace new de lo que necesita · Por qué ayuda a mantener: Cualquier pieza se reemplaza o se prueba sola
- Práctica: Datos inmutables · Cómo la aplicamos: La Solicitud es un record: cambiar de estado crea una nueva · Por qué ayuda a mantener: Sin estados a medias ni efectos ocultos
- Práctica: Errores en un solo lugar · Cómo la aplicamos: Excepciones del dominio y un manejador global · Por qué ayuda a mantener: Respuestas iguales, sin try/catch repetidos
- Práctica: Nombres del negocio · Cómo la aplicamos: Solicitud, enviar, TransicionNoPermitida · Por qué ayuda a mantener: El código se lee como el proceso real
- Práctica: Pruebas y convenciones · Cómo la aplicamos: Una prueba por regla, ramas cortas y commits con formato · Por qué ayuda a mantener: Cambiar sin miedo, con un historial legible

IDEAS CLAVE

- Una responsabilidad por clase
- Inyección por constructor
- Datos inmutables con records
- Errores en un solo lugar
- Nombres del negocio
- Pruebas y convenciones

QUÉ DECIR

“Un código mantenible es el que se puede cambiar sin miedo. Aplicamos seis prácticas sencillas.”

“Cada clase tiene una sola responsabilidad: el controlador solo traduce HTTP, el servicio decide y el adaptador guarda. Las dependencias llegan por el constructor, así cualquier pieza se puede reemplazar o probar sola. Los datos son inmutables: cambiar de estado crea una solicitud nueva, en lugar de modificar la que había.”

“Los errores se manejan en un solo lugar, los nombres son los del negocio, en español, y cada regla tiene su prueba. El resultado es que un cambio de regla toca un archivo, y una prueba nos dice si rompimos algo.”

SI PREGUNTAN

¿Esto no hace el código más largo?

Un poco, pero cada pieza es corta y se entiende sola; el costo se paga solo en el primer cambio.

PARA PASAR

“Esto es lo que aprendimos de los fundamentos.”

2 · Arquitectura

Microservicios, puertos y adaptadores, y un contrato por eventos.

6. El sistema de un vistazo

EN PANTALLA · CAPAS DEL SISTEMA

- Experiencia: Persona: Usa el navegador; arka-lite-ui: React + nginx · proxy /api
- Servicios: servicio-solicitudes: El trámite y sus estados; Evento SolicitudEnviada: Por HTTP, con su eventoId; servicio-notificaciones: Los avisos, sin duplicados
- Datos: db-solicitudes: PostgreSQL 17 · solicitud y evento; db-notificaciones: PostgreSQL 17 · evento_procesado
- Plataforma: Compose o Kubernetes: Dónde corre; GitHub Actions: Prueba y publica; Loki y Grafana: Logs en un solo lugar

IDEAS CLAVE

- La UI es la puerta de entrada
- Dos servicios, cada uno dueño de su base
- Se hablan con un evento por HTTP
- La plataforma corre, entrega y observa

QUÉ DECIR

"Este es el sistema por capas. Arriba, la experiencia: la persona usa la interfaz, y la interfaz no guarda nada, solo llama a los servicios por un proxy."

"En el medio, los dos microservicios. Solicitudes lleva el trámite; notificaciones crea los avisos. No comparten base: cada uno es dueño de sus datos, y se comunican con un evento por HTTP."

"Y abajo, la plataforma que lo sostiene: dónde corre, cómo se prueba y publica cada cambio, y dónde vemos los logs. Cada capa tiene una sola responsabilidad, y por eso cada una se puede cambiar sin romper las otras."

SI PREGUNTAN

¿Por qué no comparten una sola base de datos?

Para que cada servicio sea dueño de sus datos y pueda cambiar sin romper al otro.

PARA PASAR

"¿Y cómo se conectan esas piezas entre sí?"

7. Quién habla con quién, y cómo

EN PANTALLA · TABLA DE TRES COLUMNAS

- Conexión: Navegador -> UI · Cómo se habla: HTTP a localhost:8088; nginx sirve la página · Por qué así: El navegador nunca conoce las direcciones internas
- Conexión: UI -> servicios · Cómo se habla: nginx reenvía /api/solicitudes y /api/notificaciones · Por qué así: Cambiar una dirección no obliga a tocar la UI
- Conexión: solicitudes -> notificaciones · Cómo se habla: POST del evento SolicitudEnviada en JSON,

desde el relevador · Por qué así: Si notificaciones cae, solicitudes sigue funcionando

- Conexión: Servicio -> su base · Cómo se habla: JDBC a db-solicitudes:5432 o db-notificaciones · Por qué así: Cada servicio es dueño de sus datos
- Conexión: Cómo se encuentran · Cómo se habla: Por nombre: la red de Compose o un Service de Kubernetes · Por qué así: Sin IPs fijas: funciona igual en cualquier ambiente
- Conexión: Seguimiento · Cómo se habla: El traceId viaja en la cabecera de cada llamada HTTP · Por qué así: Una sola búsqueda en Grafana sigue todo el recorrido

IDEAS CLAVE

- El navegador solo conoce la UI
- nginx reenvía /api al servicio
- El evento viaja después, desde el relevador
- Cada servicio habla solo con su base
- Se encuentran por nombre, no por IP
- El traceId acompaña cada llamada

QUÉ DECIR

"Esta tabla resume todas las conexiones. El navegador solo conoce la interfaz. La interfaz no llama directo a los servicios: nginx recibe todo lo que empieza con /api y lo reenvía al servicio que corresponde."

"Entre servicios, solicitudes manda el evento a notificaciones con un POST en JSON, pero no en el momento de la petición: lo hace el relevador, después. Por eso, si notificaciones se cae, solicitudes sigue funcionando."

"Cada servicio habla solo con su propia base. Y nadie usa direcciones IP: se encuentran por nombre, gracias a la red de Docker Compose o a los Services de Kubernetes. En cada llamada viaja además un traceId, que nos permite seguir el recorrido completo en los logs."

SI PREGUNTAN

¿Por qué no se llaman directamente por localhost?

Porque dentro de un contenedor localhost es el mismo contenedor; por eso cada servicio se busca por su nombre.

PARA PASAR

"Hay dos formas de hablarse, y usamos las dos."

3 · Base de datos

Una base por servicio, un esquema que se cuida solo y datos consistentes.

8. Del procedimiento almacenado al ORM

EN PANTALLA · DOS MANERAS, LADO A LADO

- Con procedimientos almacenados: La regla vive en la base: EXEC sp_enviar; Cambiar una regla es cambiar la base; Para probar hace falta la base con datos; Atado a un motor: T-SQL o PL/SQL
- Con un ORM: JPA + Hibernate: La regla vive en Java, en el dominio; Hibernate genera el SQL de guardar y buscar; Las reglas se prueban sin base, en milisegundos; Cambiar de motor no toca las

reglas

IDEAS CLAVE

- Con SP, la regla vive en la base
- Con ORM, la regla vive en el código
- Hibernate escribe el INSERT, UPDATE y SELECT
- No es todo o nada: el SQL pesado puede seguir en la base

QUÉ DECIR

"Muchos de ustedes trabajan con procedimientos almacenados: la regla del negocio vive en la base, y la aplicación solo hace EXEC con parámetros. Funciona, pero cambiar una regla es cambiar la base, y probarla exige una base con datos."

"En ARKA-Lite usamos un ORM, que significa mapeo objeto-relacional. En Java el estándar se llama JPA y la herramienta que lo hace es Hibernate. Una clase Java se marca como entidad y queda asociada a una tabla: cada campo es una columna. Cuando guardamos o buscamos, Hibernate escribe el INSERT, el UPDATE o el SELECT por nosotros."

"Ojo: no es todo o nada. Las reglas del negocio pasan al código, donde se prueban en milisegundos, y lo que de verdad es trabajo de base de datos, como reportes pesados, puede seguir en SQL."

SI PREGUNTAN

¿Y el rendimiento? ¿El SQL que genera Hibernate es bueno?
Para guardar y buscar por llave, sí. Para consultas complejas se puede escribir SQL propio; además, los índices y constraints siguen siendo trabajo de la base.

PARA PASAR

"¿Y cómo llega, paso a paso, una línea de Java hasta PostgreSQL?"

---

9. Cómo el código llega a la base

EN PANTALLA · FLUJO PASO A PASO

- 1. Dominio: Pide guardar la solicitud, sin SQL.
- 2. Spring Data: save() y findById(), sin escribir SQL.
- 3. Hibernate: Objeto -> INSERT o UPDATE.
- 4. JDBC + pool: Reutiliza conexiones con usuario y clave.
- 5. PostgreSQL: Ejecuta y confirma la transacción.

IDEAS CLAVE

- El dominio pide guardar, sin SQL
- Spring Data ofrece save y findById
- Hibernate convierte el objeto en SQL
- JDBC y un pool de conexiones hablan con PostgreSQL

Comparación: Como un traductor: el código habla Java, la base habla SQL, y el ORM traduce en los dos sentidos.

QUÉ DECIR

“Sigamos una línea de código. El dominio dice: guardar esta solicitud. No sabe nada de SQL. Spring Data nos da métodos listos, como save o findById, sin escribir una sola consulta.”

“Por debajo, Hibernate convierte el objeto en un INSERT o un UPDATE. Y el driver JDBC, con un pool de conexiones que las reutiliza, como el que ya conocen de cualquier aplicación, se lo manda a PostgreSQL, que ejecuta y confirma la transacción.”

“La dirección de la base, el usuario y la clave no están en el código: llegan por variables de entorno.”

PARA PASAR

“¿Y dónde se configura todo eso?”

---

10. Guardar primero, avisar después

EN PANTALLA · FLUJO PASO A PASO

- 1. Enviar: Solicitud ENVIADA + evento pendiente, en una transacción.
- 2. Bandeja de salida: El evento espera en la tabla evento.
- 3. Relevador: Cada 5 s entrega los pendientes, hasta 3 intentos.
- 4. Notificaciones: Lo recibe y el evento queda procesado.
- Si si falla en el paso 3: Sigue pendiente: Mismo eventoId; se reintenta en el siguiente ciclo.

IDEAS CLAVE

- Solicitud ENVIADA y evento pendiente, juntos
- Si una falla, no se guarda ninguna
- El relevador entrega cada 5 s, con 3 intentos
- Si falla, sigue pendiente con el mismo eventoId

Comparación: Como dejar una carta en la bandeja de salida: aunque el mensajero no pase hoy, la carta no se pierde.

QUÉ DECIR

“Este es el problema de la consistencia. Si al enviar una solicitud llamáramos directamente a notificaciones y justo estuviera caído, la solicitud quedaría enviada y nadie se enteraría.”

“La solución se llama patrón outbox, o bandeja de salida. Al enviar, guardamos la solicitud y un evento pendiente en la misma transacción: o se guardan las dos cosas, o ninguna. La respuesta le llega a la persona enseguida.”

“Después, un relevador revisa la bandeja cada cinco segundos y entrega los pendientes, con hasta tres intentos. Si aun así falla, el evento sigue pendiente, con el mismo número, para el siguiente ciclo.”

SI PREGUNTAN

¿Que el envío responda bien significa que el aviso ya existe?

No. Significa que la solicitud y el evento quedaron guardados; el aviso llega después.

PARA PASAR

“Si se reintenta, ¿no puede llegar el aviso dos veces?”

4 · Contenedores y Kubernetes

El mismo sistema en cualquier computadora y un clúster que se repara solo.

11. ¿Qué es Docker y qué es Kubernetes?

EN PANTALLA · DOS MANERAS, LADO A LADO

- Docker: la caja: Empaqueta la app con Java y todo lo que usa; Imagen: un paquete que no cambia; Corre igual en una laptop y en un servidor; Compose levanta varias cajas juntas
- Kubernetes: el administrador: Decide dónde corre cada caja; Si una se cae, levanta otra; Le da a cada servicio un nombre fijo; Puede correr más copias si hay carga

IDEAS CLAVE

- Docker: la caja con la app y todo lo que necesita
- Imagen: el paquete; contenedor: el paquete corriendo
- Kubernetes: arranca, vigila y reinicia las cajas
- Compose hace lo mismo en una sola computadora

Comparación: Como el transporte de carga: Docker es el contenedor estándar y Kubernetes, el puerto que los acomoda, los vigila y reemplaza el que se daña.

QUÉ DECIR

"Dos palabras que se escuchan mucho. Docker resuelve el clásico "en mi máquina funciona": empaqueta la aplicación con Java y todo lo que necesita en una imagen, un paquete que no cambia. Cuando esa imagen corre, se llama contenedor, y corre igual en una laptop o en un servidor."

"Kubernetes es el siguiente nivel: no empaqueta, administra. Decide dónde corre cada contenedor, lo vigila, y si uno se cae, levanta otro. Piénsenlo como un puerto de carga: Docker es el contenedor estándar y Kubernetes es el puerto que los acomoda y reemplaza el que se daña."

"En una sola computadora usamos Docker Compose; en un clúster, Kubernetes. La misma imagen sirve en los dos."

SI PREGUNTAN

¿Docker es una máquina virtual?

No. Es más liviano: comparte el sistema operativo y solo aísla la aplicación y lo que necesita.

PARA PASAR

"Veamos el sistema completo levantado con Docker."

5 · CI/CD

Integración y entrega continua: cada cambio se prueba y se empaqueta solo.

12. Cada cambio se prueba solo

EN PANTALLA · LAS ETAPAS DEL PIPELINE

- 1. Push o PR: Alguien sube un cambio.
- 2. Compilar: Java 26 y Maven, un job por servicio.
- 3. Probar: mvn verify con PostgreSQL de servicio.
- 4. Arrancar: Levanta el JAR y espera ver CAM-002.
- 5. Imagen: Se construye con Docker.
- 6. Publicar: En GHCR, con el SHA del commit.

IDEAS CLAVE

- Cada push o Pull Request dispara el pipeline
- Compila y prueba cada servicio con su base
- Comprueba que el servicio realmente arranca
- Publica la imagen con el SHA del commit

QUÉ DECIR

"Con GitHub Actions, cada vez que alguien sube un cambio o abre un Pull Request, se dispara el pipeline. Por cada servicio prepara Java y una base PostgreSQL, compila y corre todas las pruebas."

"Luego hace algo que muchos pipelines no hacen: arranca el servicio de verdad y espera a ver los datos iniciales. Así detectamos errores de configuración que las pruebas unitarias no ven."

"Si todo pasa, construye la imagen y la publica etiquetada con el código del commit. Cada imagen se puede rastrear hasta el cambio exacto que la produjo."

SI PREGUNTAN

¿Publicar la imagen la despliega en Kubernetes?
No. Publicar deja la imagen lista; desplegar es otro paso, con kubectl apply.

PARA PASAR

"El pipeline funciona porque todo entra por el mismo camino."

6 · Observabilidad

Logs centralizados, trazas entre servicios y salud de cada componente.

13. Todos los logs, en un solo lugar

EN PANTALLA · FLUJO PASO A PASO

- 1. Servicios: Logs JSON con traceId y eventoId.
- 2. Promtail: Los recoge de cada contenedor.
- 3. Loki: Los guarda, etiquetados por servicio.
- 4. Grafana: Una búsqueda para todo el sistema.

IDEAS CLAVE

- Cada servicio escribe sus logs en JSON (ECS)

- Promtail los recoge de todos los contenedores
- Loki los guarda por servicio
- Grafana los busca por traceId o eventoId

Comparación: Como la caja negra de un avión: registra lo que pasó, para revisarlo después.

QUÉ DECIR

"Con dos servicios, cada uno tiene sus propios logs. Si algo falla, no queremos entrar contenedor por contenedor a buscar."

"Cada servicio escribe sus logs en JSON, con un formato estándar. Promtail los recoge de todos los contenedores, Loki los guarda etiquetados por servicio y Grafana nos deja buscarlos en un solo lugar."

"Las llamadas entre servicios llevan el traceId, así que con una sola búsqueda vemos el viaje de un evento: cuándo lo publicó el relevador y cuándo lo consumió notificaciones."

QUÉ HACER (NO SE DICE)

- Mostrar: Grafana ? Explore ? Loki: {service="servicio-notificaciones"} |= "evento". Debe verse: Las líneas "Consume evento" y "aviso creado" con el número del evento.

PARA PASAR

"Los logs cuentan qué pasó. La salud dice cómo estamos ahora."

7 · Documentación

Contratos que salen del código y documentación que cambia con él.

14. Contratos vivos con Swagger

EN PANTALLA · CAPTURAS REALES

- Captura de Swagger de solicitudes (localhost:8080/swagger-ui/index.html)

IDEAS CLAVE

- Cada servicio publica su contrato OpenAPI
- Swagger UI lo muestra y permite probarlo
- Un endpoint nuevo aparece solo
- La UI enlaza los dos Swagger

Comparación: Como el menú de un restaurante: dice qué hay, qué lleva cada plato y qué te van a traer.

QUÉ DECIR

"Un contrato de API es como el menú de un restaurante: dice qué se puede pedir, qué lleva cada pedido y qué te van a devolver."

"Cada servicio publica su contrato con el estándar OpenAPI, usando springdoc, y Swagger lo muestra como una página donde se puede ver cada endpoint, sus códigos de respuesta y probarlo en vivo."

"Lo importante para el staff: se genera desde el código. Cuando agregamos un endpoint, aparece solo, y nunca queda desactualizado. Un equipo que consume la API no tiene que pedirnos un documento."

QUÉ HACER (NO SE DICE)

- Mostrar: En la UI, pestaña Salud ? Swagger · solicitudes, y ejecutar GET /solicitudes. Debe verse: La lista en JSON con código 200.

PARA PASAR

“Y lo que no sale del código, lo documentamos junto al código.”

8 · La UI como laboratorio

No es parte del entregable: es el banco de pruebas para ver cómo se comporta el sistema, sobre todo cuando algo falla.

15. Si notificaciones se cae

EN PANTALLA · QUÉ PASA CUANDO FALLA

- Falla: notificaciones está caído: docker compose stop servicio-notificaciones
- 1. [0 s] Enviar: Responde 200: solicitud ENVIADA y evento guardado.
- 2. [cada 5 s] Reintenta: 3 intentos por ciclo; el evento sigue pendiente.
- 3. [en la UI] Se ve: “Notificaciones no responde” y 1 en camino.
- 4. [al volver] Entrega: Mismo eventoId: se crea el aviso.
- Resultado: 1 aviso, 0 perdidos: Sin intervención y sin duplicados.

IDEAS CLAVE

- Enviar responde bien aunque notificaciones esté caído
- El evento queda pendiente en la bandeja
- El relevador reintenta cada 5 segundos
- Al volver, llega un solo aviso

QUÉ DECIR

“Detenemos notificaciones a propósito. En la pestaña Salud aparece en DOWN.”

“Ahora enviamos una solicitud. La respuesta es buena: la solicitud queda enviada y su evento queda guardado. La persona no ve ningún error, porque no depende de que el otro servicio esté arriba.”

“En Enviadas aparece un aviso: notificaciones no responde y el evento espera en la bandeja de salida. El relevador lo intenta cada cinco segundos y falla, pero no lo pierde. Cuando levantamos notificaciones, en el siguiente ciclo lo entrega con el mismo número, y aparece exactamente un aviso.”

QUÉ HACER (NO SE DICE)

- Antes: docker compose stop servicio-notificaciones. Mostrar Salud en DOWN.
- Crear y enviar DEMO-01; mostrar en Enviadas el aviso amarillo “Notificaciones no responde” y “1 en camino”. No hace falta recargar: la pantalla se actualiza sola y el indicador de servicios vuelve a 2 de 2.
- docker compose start servicio-notificaciones y esperar unos segundos: el evento pasa a entregado y aparece 1 aviso.
- Mostrar: Detener notificaciones, enviar DEMO-01, mirar Enviadas y volver a iniciar notificaciones. Debe verse: Evento en camino mientras está caído; al volver, entregado y 1 aviso.

SI PREGUNTAN

¿Cuánto tiempo puede esperar un evento pendiente?

Indefinidamente: está en la base. Se entrega en el primer ciclo después de que notificaciones vuelve.

PARA PASAR

“Ese es el caso más importante, pero no el único.”

Valor y cierre

16. El valor agregado

EN PANTALLA · TABLA DE TRES COLUMNAS

- Pilar: Resiliencia · Qué construimos: Outbox, reintentos e idempotencia · Valor para el equipo: 0 avisos perdidos o duplicados ante caídas
- Pilar: Reproducibilidad · Qué construimos: Docker, Compose y Kubernetes · Valor para el equipo: 1 comando y la misma imagen en todo ambiente
- Pilar: Calidad · Qué construimos: 67 pruebas y pipeline por PR · Valor para el equipo: Errores detectados en minutos, antes de integrar
- Pilar: Trazabilidad · Qué construimos: Logs centralizados, traceId e imagen por SHA · Valor para el equipo: Diagnóstico en minutos y versiones rastreables
- Pilar: Contratos y diseño · Qué construimos: OpenAPI generado y puertos y adaptadores · Valor para el equipo: Integraciones claras y tecnología intercambiable

IDEAS CLAVE

- Resiliencia ante fallas parciales
- Entornos reproducibles
- Calidad en cada cambio
- Trazabilidad de punta a punta

QUÉ DECIR

“Si tuviéramos que resumir el valor en una lámina, sería esta. Construimos resiliencia: ante una caída parcial no se pierde ni se duplica un aviso. Reproducibilidad: un comando y la misma imagen en cualquier ambiente.”

“Calidad: sesenta y siete pruebas en cada Pull Request. Trazabilidad: logs centralizados con traceId e imágenes rastreables a su commit. Y contratos claros con un diseño que permite cambiar tecnología sin tocar el negocio.”

“Nada de esto es exclusivo de ARKA-Lite: son prácticas que cualquier equipo puede adoptar desde mañana.”

PARA PASAR

“Si nos llevamos solo tres ideas, que sean estas.”

17. Lo que nos llevamos

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 0 avisos perdidos o duplicados
- Métrica: 67 pruebas en cada cambio
- Diseñar para la falla: Guardar primero, reintentar e ignorar duplicados. Valor: Sistemas que se recuperan solos.
- Automatizar la confianza: Pruebas, pipeline, imágenes y estado declarado. Valor: Cambiar rápido sin miedo a romper.
- Hacer visible el sistema: Logs, trazas, salud, Swagger y la UI laboratorio. Valor: Diagnosticar y explicar en minutos.

IDEAS CLAVE

- Diseñar para la falla
- Automatizar la confianza
- Hacer visible el sistema

QUÉ DECIR

"Tres ideas. Diseñar para la falla: en un sistema distribuido, algo siempre va a fallar, y la diferencia está en cómo reacciona."

"Automatizar la confianza: pruebas, pipeline e infraestructura declarada nos permiten cambiar rápido sin miedo."

"Y hacer visible el sistema: lo que no se ve no se puede diagnosticar ni explicar."

PARA PASAR

"Gracias por su atención."

18. Gracias

EN PANTALLA

- Un laboratorio que cumplió su objetivo: aprender.
- Proyecto final · Upskilling Backend Java

QUÉ DECIR

"ARKA-Lite fue un laboratorio, y cumplió su objetivo: aprender. Empezamos con un endpoint y terminamos con dos microservicios con su base de datos, empaquetados, desplegados, observados y documentados, que saben comportarse cuando algo falla."

"Muchas gracias por acompañarnos. Con gusto respondemos sus preguntas."